Update 8: The Dual QP Pool on libfabric

Nathan Gopee
MS Computer Science Candidate, SUNY New Paltz
gopeen1@newpaltz.edu
Advisors: Dr. Wafi Danesh & Dr. Ashley Suchy

October 2026 (draft: method and build complete, cluster numbers pending)

Contents

1 Summary **1**

2 Why libfabric **1**

3 Mapping the pool onto libfabric **2**

4 One harness, two libraries **2**

5 Provider behavior found while building **2**

6 Experiments **3**
6.1 E1. Hot-path latency by stack 3
6.2 E2. Cold bandwidth 3
6.3 E3. Head-of-line blocking, single vs dual endpoint 3
6.4 E4. libfabric configuration matrix 4
6.5 E5. Completion signaling 4
6.6 E6. Application level 4

7 What the comparison can and cannot show **4**

8 Next steps **4**

1 Summary

Update 7 placed libscuffedrdma among the libfabric, UCX and DOCA stacks and noted that libfabric's endpoint model matches the hot/cold split directly. This update builds that version and sets up the comparison against the libibverbs pool.

What exists now:

- `libscuffedfabric`: the dual pool on libfabric. Each class (hot, cold) gets its own endpoint, CQs and, if wanted, its own provider and endpoint type (`FI_EP_MSG` or `FI_EP_RDM`). Classes with the same provider and type share one fabric, domain and memory registration.
- `libscuffedverbs`: the same C API implemented on raw libibverbs RC queue pairs. The benchmark source is shared, so `sf_bench` and `sv_bench` differ only in the library they link.

- Benchmarks for hot-path latency under five completion-signaling mechanisms, cold bandwidth, and a mixed hot/cold head-of-line test, with per-post instruction counts and CPU pinning.
- A Python binding and a blob channel over the pool, which the vLLM connector uses as its transport.

Everything below was built and exercised end to end on libfabric 1.17's `tcp`, `tcp;ofi_rxm` and `shm` providers. Those runs check correctness only. The comparison itself (verbs provider and native verbs on SoftRoCE between Cerberus and Chimera, and on ConnectX-4 between the towers) is notebook 03 in cluster mode, and its tables are empty here on purpose.

2 Why libfabric

Three things point the same way.

1. **The endpoint model fits the design.** libfabric exposes connected (`MSG`) and reliable-unconnected (`RDM`) endpoints, completion counters as a lighter alternative to CQs, selective completion, and wait objects on CQs [1, 2, 3]. Hot traffic wants the cheapest possible completion; cold traffic wants a deep queue and a thread that sleeps. With libibverbs both have to be built by hand. libfabric exposes them as configuration.
2. **GPFS runs on it.** IBM Storage Scale uses libfabric underneath its RDMA path. A cold class on the same stack the file system already uses is the natural route for KV spill to a GPFS tier (Update 6).
3. **The stack choice is measurable and matters.** Smith's MUG'23 comparison concludes that libfabric and UCX "both provide very similar performance benefits" [5]. EPCC's ARCHER2 study found little difference in OSU micro-benchmarks but large application-level differences, concentrated in `MPI_Alltoallv`, and could not reproduce them with the micro-benchmarks [6]. Micro-benchmarks alone do not settle a stack comparison. That is why this update measures under mixed traffic and at the connector level, not only with ping-pong.

3 Mapping the pool onto libfabric

Concept	**libibverbs** (`libscuffedverbs`)	**libfabric** (`libscuffedfabric`)
Class channel	RC QP per class	endpoint per class, `FI_EP_MSG` or `FI_EP_RDM`
Shared resources	one PD, one MR	one fabric + domain + `fi_mr_reg` when providers and types match
Hot completion	busy-polled send CQ	busy-polled CQ, or `fi_cntr` with `FI_SELECTIVE_COMPLETION`
Cold completion	send CQ + completion channel	CQ with `FI_WAIT_UNSPEC`, `fi_cq_sread`
Notification	`RDMA_WRITE_WITH_IMM` (32-bit imm)	`fi_writedata`, `cq_data_size` bytes (4 on verbs, 8 on tcp/shm)
Receive slots	posted recv WRs	posted `fi_recv`, required under `FI_RX_CQ_DATA`
Remote addressing	virtual address + rkey	virtual address or offset (`FI_MR_VIRT_ADDR`), provider or requested key
Connection setup	QPN/PSN/GID over a TCP side channel	MSG: passive endpoint, connect param selects the class; RDM: `fi_getname` + `fi_av_insert`
Remote-write counting	none	`FI_RMA_EVENT` counter, endpoint or MR bound
Progress	NIC / rxe kernel thread	provider-defined: auto (verbs) or manual (`ofi_rxm`, `shm`)

Table 1: The same pool in both libraries. Both export `scuffed_fabric.h`.

The mixed configuration from Update 7, a hot class on an RDM endpoint and a cold class on a connected one, is a run-time option (`--hot-prov 'verbs;ofi_rxm' --hot-ep rdm --cold-prov verbs --cold-ep msg`). It needs two domains, so the arena is registered twice. On verbs, `ofi_rxm` runs its own eager and rendezvous protocol on top of RC [4], so this configuration puts a protocol layer back on the hot path. Whether that costs more than the RDM endpoint's connection handling saves is one of the questions the matrix in Section 6 answers.

4 One harness, two libraries

`sv_bench` is `sf_bench.c` compiled against `libscuffedverbs`. Payload offsets, warmup, window, timing (`CLOCK_MONOTONIC_RAW`), percentile code and JSON schema are identical. The verbs library checks `wc.status` on every completion and treats error and flush completions as errors.

That check is a correction carried into this update. The Python pool from Update 4 did not check completion status, so failed or flushed writes counted as fast completions. The 8 μs cross-node figure reported earlier included 8 MB writes that "completed" in about 8 μs, which 10GbE cannot do. The pool now raises on a bad status, and notebook 01 re-runs that benchmark. Until then, that figure is withdrawn. The scuffedQuant ratio quoted as 4× against FP16 was against FP32. With bit-packed indices and a 256-bit QJL sketch at head_dim 64 it is 2× against FP16.

5 Provider behavior found while building

These come from libfabric 1.17's `tcp`, `tcp;ofi_rxm` and `shm` providers in a two-core container. They shaped the library and the benchmark. Whether the verbs provider shares any of them is part of the cluster run.

- `tcp` **needs target-side CQ reads despite** `FI_PROGRESS_AUTO`**.** With the target not reading the cold endpoint's CQ, a 1 MB write stream stalled after about three writes (socket buffers full). The benchmark server now runs a yielding progress thread on the cold endpoint for every provider. On verbs it polls an empty CQ.
- **Remote-write counters are not uniform.** `tcp` advertises `FI_RMA_EVENT` but did not increment an endpoint-bound `FI_REMOTE_WRITE` counter. Registering the MR with `FI_RMA_EVENT` was accepted, but `fi_mr_enable` returned `-FI_ENOSYS`. `ofi_rxm` does not offer the capability. `shm` counts through the endpoint. The library binds both ways, and the benchmark probes one write in each direction before trusting a counter.
- **Connection-request info drops optional caps.** On the passive side, the `fi_info` delivered with `FI_CONNREQ` lacked `FI_RMA_EVENT` even though the listener requested it. The library copies it back before creating the endpoint.
- `ofi_rxm` **connects on first use.** The initiator's first write spins on `-FI_EAGAIN` until the target makes progress. Posting is now bounded (60 s), so an idle peer becomes an error instead of a hang.
- `shm` **does not advertise write-after-write ordering** for RMA, so the sentinel mechanism is skipped there.
- **With one endpoint the target cannot tell classes apart.** The single-endpoint baseline therefore measures exactly the coupling the dual pool removes. It is not a mode that could be made class-aware.

6 Experiments

All runs: Cerberus as client, Chimera as server, SoftRoCE on the 10GbE link, hot and cold threads pinned to separate cores away from the NIC interrupts. Repeat on Tower 1 and Tower 2 over ConnectX-4 for the hardware-RoCE column. Every number in the tables traces to a JSON file under `benchmarks/results/fabric/` that records the exact server and client command.

6.1 E1. Hot-path latency by stack

`sig -M cq`, one-way latency (RTT/2), 8 B to 64 KB. Notebook 03, section 1.

Stack	64 B p50 (μs)	64 B p99 (μs)	4 KB p50 (μs)	4 KB p99 (μs)
libibverbs RC (native)	*pending*	*pending*	*pending*	*pending*
libfabric verbs MSG	*pending*	*pending*	*pending*	*pending*
libfabric verbs;ofi_rxm RDM	*pending*	*pending*	*pending*	*pending*

Table 2: Hot-path latency, SoftRoCE, Cerberus ↔ Chimera.

6.2 E2. Cold bandwidth

`bw`, window 16, 4 KB to 16 MB, with and without a wait object on the cold CQ. Notebook 03, section 2.

6.3 E3. Head-of-line blocking, single vs dual endpoint

`hol`: hot 64 B and 4 KB ping-pong while the cold class streams 256 KB, 1 MB and 4 MB writes, window 8. The thesis claim holds on a stack if dual-endpoint hot p99 under load stays near its unloaded p99 while single-endpoint p99 grows with cold message size. If both grow, the bottleneck sits below the queue pair (for SoftRoCE, the rxe kernel thread), which is itself a result. Notebook 01.

Stack, endpoints	hot p99, no cold	hot p99, 1 MB cold	cold MB/s
libibverbs, single QP	*pending*	*pending*	*pending*
libibverbs, dual QP	*pending*	*pending*	*pending*
libfabric verbs MSG, single EP	*pending*	*pending*	*pending*
libfabric verbs MSG, dual EP	*pending*	*pending*	*pending*
libfabric verbs;ofi_rxm RDM, dual EP	*pending*	*pending*	*pending*

Table 3: Hot 64 B round trip (μs) under a cold stream.

6.4 E4. libfabric configuration matrix

Under the same mixed load: MSG/MSG, RDM/RDM, hot RDM + cold MSG, hot completions through `fi_cntr`, cold CQ with a wait object, and the single-endpoint baseline. Notebook 03, section 3.

6.5 E5. Completion signaling

CQ data, flag in payload, sentinel write, remote-write counter, and write followed by an 8 B read (the GPFS flush pattern), each with user-space instructions retired per post, following Papadopoulou et al.'s per-operation accounting [7]. Adopt a mechanism only if it moves p99. Notebook 02.

6.6 E6. Application level

`FabricBlobChannel` against `TcpBlobChannel` for 4 KB to 32 MB blobs, cross node, with an end-to-end acknowledgment per blob (`benchmarks/fabric/blob_bench.py`). This is the transport under the vLLM connector, so it links the micro-benchmarks to the TTFT runs in notebook 06.

7 What the comparison can and cannot show

- On SoftRoCE the verbs provider and native verbs reach the same rxe device, so their difference is libfabric's software path (dispatch, completion translation, and for RDM the `ofi_rxm` protocol). It is not a NIC difference. The ConnectX-4 runs separate software cost from a hardware path that is no longer the bottleneck.
- rxe processes packets in a kernel thread. If that thread saturates, separate QPs share it, and dual-endpoint gains shrink regardless of library. E3 shows whether that happens.
- Provider versions matter. The cluster runs record `fi_info --version` and the provider string per class in every JSON.

8 Next steps

1. Run notebooks 00 to 03 in cluster mode on Cerberus and Chimera, then on the two towers. Fill E1 to E5 from the JSON.
2. Run the vLLM connector over `FabricBlobChannel` on the verbs provider and compare TTFT and ITL against NIXL (notebook 06).
3. If hot-RDM plus cold-MSG wins E4, make it the default and route the cold class's NVMe and GPFS spill through the same libfabric domain.
4. Feed the telemetry signal dumps from these runs into the learned WFA (notebook 04).

References

[1] P. Grun, S. Hefty, S. Sur, D. Goodell, R. D. Russell, H. Pritchard, and J. M. Squyres, "A Brief Introduction to the OpenFabrics Interfaces: A New Network API for Maximizing High Performance Application Efficiency," *IEEE Hot Interconnects*, 2015.

[2] OpenFabrics Interfaces Working Group, `fi_endpoint(3)`, `fi_cq(3)`, `fi_mr(3)` man pages, libfabric 1.17. https://ofiwg.github.io/libfabric/

[3] OpenFabrics Interfaces Working Group, `fi_cntr(3)` man page, libfabric. https://ofiwg.github.io/libfabric/

[4] OpenFabrics Interfaces Working Group, `fi_rxm(7)` and `fi_verbs(7)` man pages, libfabric. https://ofiwg.github.io/libfabric/

[5] B. Smith, "Omni-Path and the Open Fabrics Interfaces," Cornelis Networks, MVAPICH User Group (MUG) 2023.

[6] M. Bareford, D. Henty, W. Lucas, and A. Turner, "OpenFabrics and UCX Performance on the ARCHER2 HPE Cray EX System," Cray User Group (CUG) 2022. Data: https://github.com/ARCHER2-HPC/performance_ofi-ucx

[7] N. Papadopoulou, L. Oden, and P. Balaji, "A Performance Study of UCX over InfiniBand," *Proc. IEEE/ACM CCGrid*, 2017.

[8] R. Zambre, "For the RDMA novice: libfabric, libibverbs, InfiniBand, OFED, MOFED?," 2018. https://www.rohitzambre.com/blog/2018/2/9/for-the-rdma-novice-libfabric-libibverbs-infiniband-ofed-mofed